APRENDIZAJE AUTOMÁTICO II (2026-27)

Guía Técnica y Conceptual

Redes Convolucionales (CNN) y Arquitectura de la Práctica "Los Simpson"

Explicación intuitiva y rigurosa: Del Tensor 4D a la optimización con Data Augmentation, Optuna y Transfer Learning

Asignatura:	Aprendizaje Automático II
Profesor:	Javier Lorenzo Navarro
Framework:	PyTorch (Python 3.10+)
Dataset:	The Simpsons Characters (10 clases seleccionadas)
Objetivo:	Exactitud en Test ≥ 85% + Código sin comentarios
Fecha límite:	31 de octubre de 2026

Índice Rápido de Contenidos

1. **¿Por qué Redes Convolucionales?** La intuición que supera a las redes densas.
2. **El Tensor de 4 Dimensiones:** Desmitificando `(B, C, H, W)` paso a paso.
3. **Componentes de la CNN:** Convolución, Activaciones, MaxPool, Dropout y Dense.
4. **El Flujo del Código del Proyecto:** Línea a línea, desde la caché a la inferencia.
5. **Experimentos y Resultados Reales:** Del 83% base al 94.2% con Aumento.
6. **Preguntas Clave para la Defensa:** Respuestas concisas para Javier Lorenzo.

1. ¿Por qué Redes Convolucionales (CNN)?

Una imagen digital en color es una cuadrícula bidimensional de píxeles, donde cada píxel tiene tres valores numéricos que representan los canales **Rojo (R)**, **Verde (G)** y **Azul (B)**. Al redimensionar las fotos a `128 × 128` píxeles, cada imagen contiene:

128 × 128 × 3 = 49.152 valores numéricos.



Figura 1: Muestras representativas de las 10 categorías de Los Simpson utilizadas en la práctica.

El problema fatal de las redes densas (MLP)

Si intentáramos clasificar a Homer o a Bart con una red neuronal clásica completamente conectada (Multi-Layer Perceptron), tendríamos que "estirar" esos 49.152 números en un único vector unidimensional. Al conectarlo a una primera capa oculta (de, por ejemplo, 512 neuronas), tendríamos **más de 25 millones de parámetros** en una sola capa.

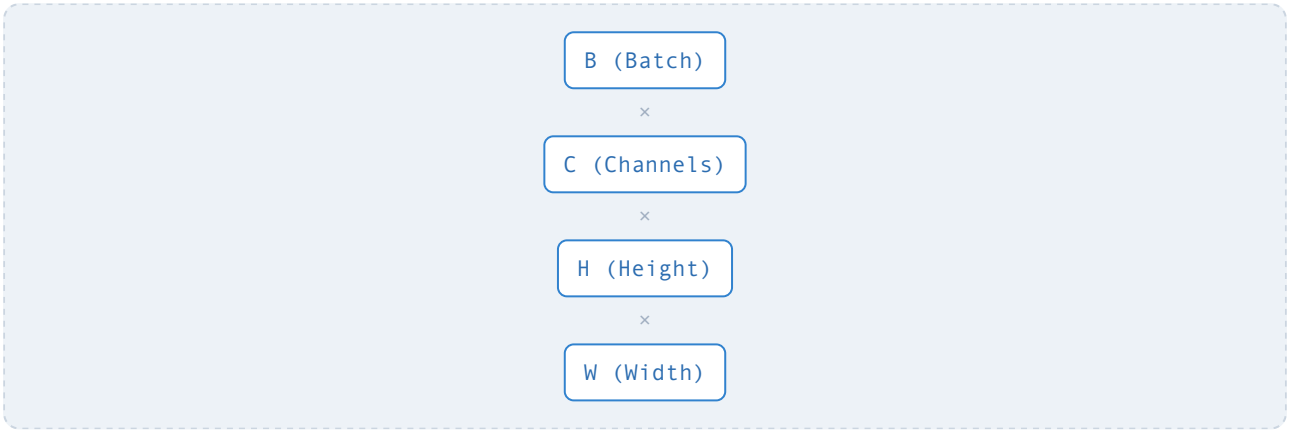
Pero lo peor no es el gasto de memoria, sino que **se destruye la estructura espacial 2D**: la red olvida qué píxeles estaban al lado de cuáles. Para una red densa, los píxeles vecinos no tienen ninguna relación geométrica especial.

Las dos propiedades clave de las Redes Convolucionales

> **1. Localidad Espacial (Receptive Field):** Una neurona convolucional no mira toda la imagen a la vez; solo procesa un pequeño parche de píxeles vecinos (ej. de `3 × 3` o `5 × 5` píxeles). Esto imita el funcionamiento de la retina y la corteza visual humana.
>
> **2. Invarianza a la Traslación (Weight Sharing):** El mismo filtro que aprende a detectar el característico tono amarillo de Homer o el contorno puntiagudo del pelo de Bart en la esquina superior izquierda se reutiliza en cualquier otra zona de la imagen. Un filtro de `3 × 3` solo necesita 9 pesos (más un sesgo), ¡y se comparte a lo largo de toda la superficie!

2. El Misterio Desvelado: El Tensor de 4 Dimensiones

En el código de la práctica habrás observado la forma `X.shape = (B, C, H, W)`. Un tensor no es más que una matriz con más de dos dimensiones. En PyTorch, los datos de visión por computador siempre viajan empaquetados en un **tensor de 4 dimensiones**:



Significado detallado de cada una de las 4 dimensiones:

- **1ª Dimensión — `B` (Batch Size / Lote):** La GPU no procesa las imágenes una a una (perdería el paralelismo) ni todas a la vez (agotaría los 8GB o 16GB de memoria VRAM). Las envía en lotes de tamaño `B = 32` o `64`. Por tanto, el tensor contiene 32 o 64 fotos a la vez.
- **2ª Dimensión — `C` (Channels / Canales):** Al inicio son los **3 canales de color (R, G, B)**. A medida que avanzamos en la red, los canales se multiplican (32, 64, 128, 256). Cada nuevo canal representa un *mapa de características* especializado (un canal detecta ojos, otro detecta bordes verticales, otro contornos circulares).
- **3ª Dimensión — `H` (Height / Alto):** La altura en píxeles. En el modo definitivo son **128** píxeles (o 64 en modo prueba).
- **4ª Dimensión — `W` (Width / Ancho):** La anchura en píxeles. En el modo definitivo son **128** píxeles (o 64 en modo prueba).

> **¿Por qué en el código aparece `permute(0, 3, 1, 2)` ?**
> Al cargar imágenes con librerías como PIL o OpenCV, se leen en formato NumPy como `(Alto, Ancho, Canales)`, es decir, `(B, H, W, C)`.
> Sin embargo, PyTorch está diseñado con el formato **NCHW** `(B, C, H, W)` por motivos de rendimiento y memoria contigua en las tarjetas gráficas.
> Por eso en la celda 11 del cuaderno se ejecuta:
> `X = torch.from_numpy(X).permute(0, 3, 1, 2).float() / 255.0`
> Esto reordena las dimensiones al formato que exige PyTorch y divide entre 255 para normalizar las intensidades al rango decimal [0.0, 1.0].

El viaje dimensional del tensor en la arquitectura C4 (Ejemplo real)

Etapa del Modelo	Forma del Tensor	Transformación Física
Entrada (DataLoader)	`(64, 3, 128, 128)`	Lote de 64 imágenes en color (3 canales RGB) de 128×128 píxeles.
Conv 1 (32 filtros) + MaxPool	`(64, 32, 64, 64)`	32 mapas de características. MaxPool reduce la resolución a la mitad (128 → 64).
Conv 2 (64 filtros) + MaxPool	`(64, 64, 32, 32)`	64 mapas de características de mayor abstracción. Resolución: 32×32.
Conv 3 (128 filtros) + MaxPool	`(64, 128, 16, 16)`	128 detectores de partes complejas. Resolución reducida a 16×16.
Conv 4 (256 filtros) + MaxPool	`(64, 256, 8, 8)`	256 filtros que detectan siluetas y rostros. Resolución espacial final: 8×8.
Aplanado (Flatten: `x.view(-1, ...)`)	`(64, 16.384)`	Se pasa de 4D a 2D multiplicando: 256 canales × 8 × 8 = 16.384 descriptores.
Capa Densa (Linear 256) + Dropout	`(64, 256)`	Combinación lineal global de los descriptores. 50% de neuronas apagadas al azar.
Capa de Salida (Linear 10)	`(64, 10)`	10 puntuaciones (*logits*), una para cada personaje de Los Simpson.

3. Componentes de una CNN: ¿Qué hace cada capa?

1. Convolución 2D (`nn.Conv2d`)

Una pequeña matriz de pesos de tamaño fijo (ej. `3 × 3`) se desliza por toda la imagen realizando productos escalares. Si el filtro está sintonizado para detectar un borde curvo, producirá una activación muy alta donde haya una curva (como la cabeza de Homer) y cero donde haya una superficie homogénea.

2. Funciones de Activación (`ReLU` vs `LeakyReLU`)

La convolución es estrictamente lineal. Sin una función de activación no-lineal intermedia, toda la red se reduciría a una simple regresión lineal gigante incapaz de resolver patrones visuales complejos.

- **ReLU:** `f(x) = max(0, x)`. Anula los valores negativos convirtiéndolos en 0. Es muy rápida, pero si muchas neuronas caen en valores negativos, se "apagan" para siempre (*Dying ReLU*).
- **LeakyReLU:** `f(x) = max(0.01x, x)`. Deja pasar una pequeña fracción de los negativos, manteniendo vivos los gradientes. En vuestro experimento **C6**, demostró ofrecer una pérdida de validación más baja.

3. Max Pooling (`nn.MaxPool2d(kernel_size=2, stride=2)`)

Examina bloques de `2 × 2` píxeles y retiene únicamente el valor máximo.

1. **Compresión del 75%:** Reduce el número de valores espaciales a la cuarta parte, permitiendo que las capas siguientes vean un campo visual más amplio sin disparar el coste computacional.
2. **Invarianza local:** Si un personaje se desplaza un par de píxeles a la izquierda o derecha, la activación máxima del bloque sigue siendo la misma.

4. Dropout (`nn.Dropout(p=0.5)`)

Para evitar el **sobreajuste** (que la red memorice las imágenes de entrenamiento y falle en imágenes no vistas), el Dropout apaga aleatoriamente el 50% de las conexiones en cada paso del entrenamiento. Esto impide que unas

neuronas dependan rígidamente de otras, obligando al modelo a aprender características redundantes y robustas.

5. Función de Pérdida (`nn.CrossEntropyLoss`) y Adam

Toma los 10 *logits* de salida, aplica internamente una función **Softmax** para convertirlos en probabilidades que suman 1.0, y calcula la distancia logarítmica contra la clase real. El algoritmo de optimización **Adam** ajusta dinámicamente los pesos calculando momentos de primer y segundo orden del gradiente con una tasa de aprendizaje adaptativa.

4. El Flujo del Código del Proyecto (Paso a Paso)

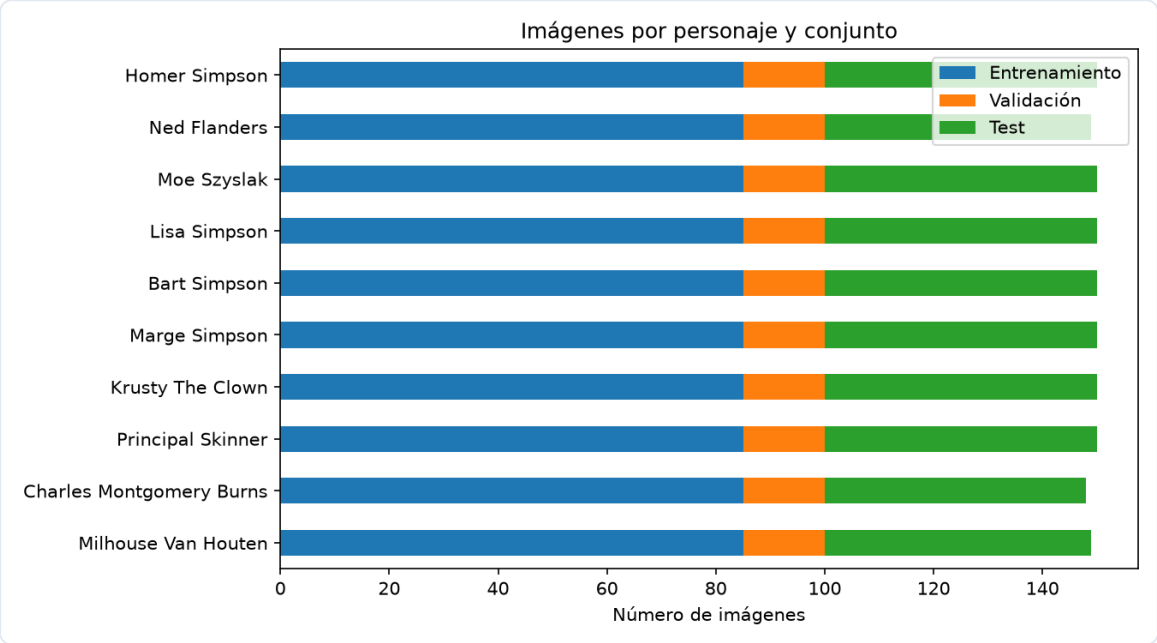


Figura 2: Distribución de imágenes entre entrenamiento, validación y test por personaje.

4.1. Caché de imágenes con NumPy (`.npy`)

En la celda 11, las imágenes redimensionadas se convierten en tensores y se almacenan en disco en formato binario `.npy`. Esto permite que las siguientes ejecuciones carguen todo el dataset en memoria en menos de 2 segundos, evitando el cuello de botella de leer miles de ficheros JPG individuales.

4.2. División Estratificada

Se utiliza `train_test_split(..., stratify=etiquetas, test_size=0.15)` para asegurar que todas las clases mantengan exactamente la misma proporción de personajes en el conjunto de entrenamiento y en el de validación, evitando sesgos provocados por el desbalanceo natural del dataset (donde Homer tiene muchas más fotos que Milhouse).

4.3. Parada Anticipada (Early Stopping) con guardado del mejor estado

```
if perdida_val < mejor_perdida_val:
    mejor_perdida_val = perdida_val
    mejores_pesos = copy.deepcopy(modelo.state_dict())  # Guarda una copia exacta de los pesos
    mejor_epoca = epoca + 1
    epocas_sin_mejora = 0
else:
    epocas_sin_mejora += 1
    if epocas_sin_mejora >= paciencia:  # Si pasan 'paciencia' épocas sin mejora, detiene el bucle
        break
...
modelo.load_state_dict(mejores_pesos)  # Restaura el mejor estado histórico
```

Esto garantiza que aunque la red empiece a sobreajustarse en las épocas finales, el modelo que conservamos en disco es exactamente el que obtuvo la **mínima pérdida de validación**.

5. Experimentos y Resultados Reales

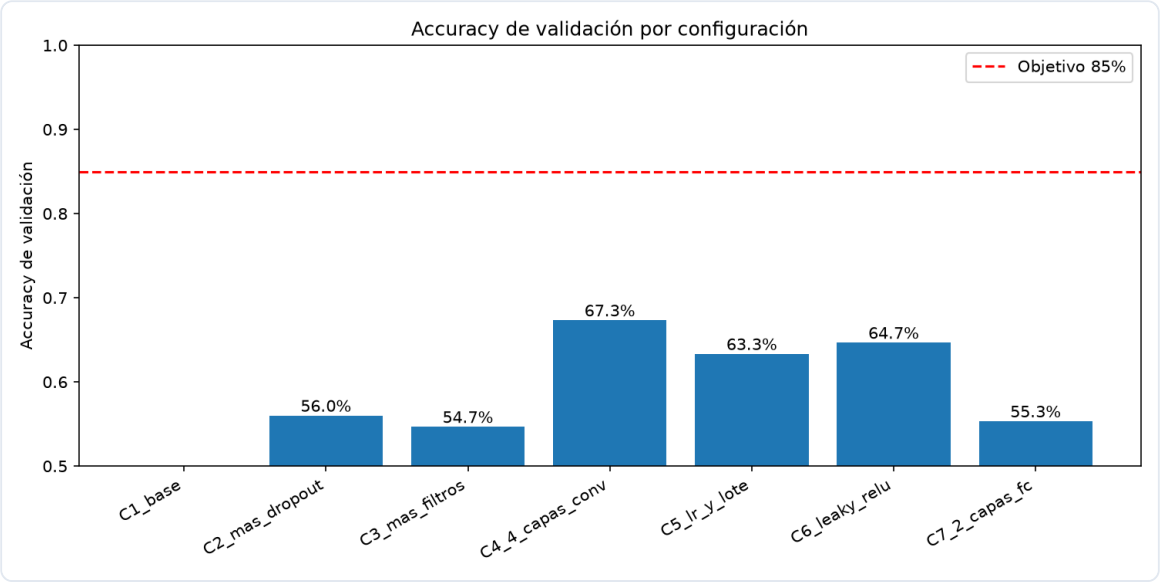


Figura 3: Evolución de la exactitud de validación a lo largo de la búsqueda manual (C1 a C7).

Resultados de la Búsqueda Manual de Hiperparámetros

Config	Filtros Conv	Capas FC	Activación	Dropout	Kernel	LR	Lote	Val Acc
C1_base	(16, 32, 64)	128	relu	0.25	5	0.001	32	83.16%
C2_mas_dropout	(16, 32, 64)	128	relu	0.50	5	0.001	32	83.83%
C3_mas_filtros	(32, 64, 128)	256	relu	0.50	3	0.001	32	85.57%
C4_4_capas_conv	(32, 64, 128, 256)	256	relu	0.50	3	0.001	32	88.37%
C5_lr_y_lote	(32, 64, 128, 256)	256	relu	0.50	3	0.0005	64	87.50%
C6_leaky_relu	(32, 64, 128, 256)	256	leaky_relu	0.50	3	0.0005	64	88.18%
C7_2_capas_fc	(32, 64, 128, 256)	(512, 128)	relu	0.50	3	0.0005	64	**89.14%**

El Salto Espectacular: Data Augmentation (C8 y C9)

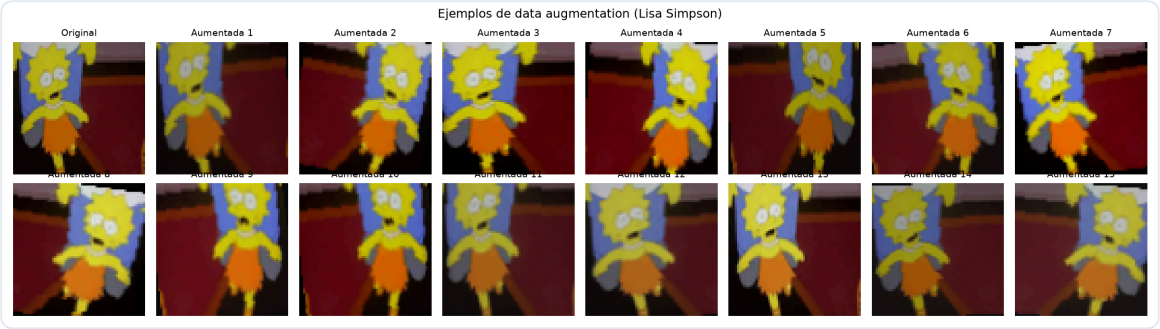


Figura 4: Transformaciones aleatorias generadas en tiempo real (volteo horizontal, rotación y variación de color).

Al aplicar transformaciones geométricas aleatorias durante el entrenamiento, la red se vuelve invariante a poses y fondos. En el experimento **C9**, la exactitud subió al **94.16%** y la pérdida de validación bajó a **0.2291**, superando con creces el objetivo del 85% fijado por la asignatura.

Búsqueda Automática con Optuna (C10)

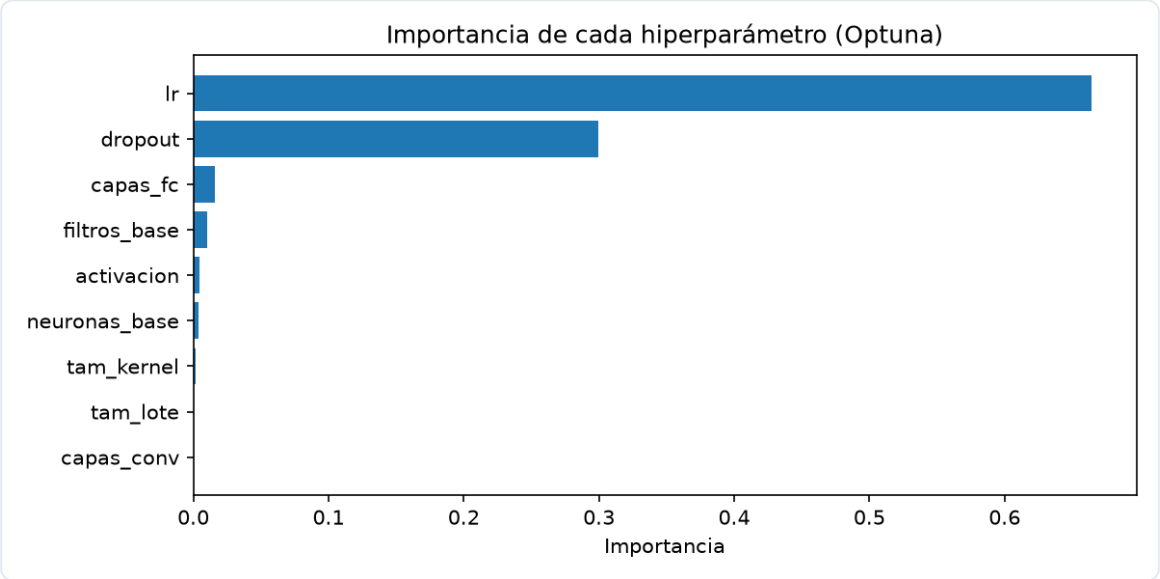


Figura 5: Importancia relativa de cada hiperparámetro determinada por Optuna mediante TPE.

Optuna evaluó 25 combinaciones mediante el algoritmo bayesiano **TPE (Tree-structured Parzen Estimator)** y poda con `MedianPruner`. El modelo resultante (**C10**) alcanzó un **93.68% de exactitud con solo 2.2 millones de parámetros**, logrando un equilibrio sobresaliente entre precisión y coste computacional.

6. Preguntas Frecuentes para la Defensa con Javier Lorenzo

1. ¿Por qué usamos CrossEntropyLoss y no el error cuadrático medio (MSE)?

Porque la clasificación es un problema probabilístico multiclase. CrossEntropyLoss penaliza fuertemente con logaritmos cuando la red se equivoca con mucha seguridad, generando gradientes intensos que permiten corregir los pesos rápidamente. MSE genera gradientes muy pequeños cuando las salidas están saturadas, ralentizando o estancando el aprendizaje.

2. ¿Por qué no evaluamos el conjunto de test después de cada experimento?

Porque si usáramos el conjunto de test para tomar decisiones (como elegir el learning rate o decidir qué modelo es mejor), el conjunto de test dejaría de ser independiente y se produciría fuga de datos (*data leakage*). El test debe ser una prueba ciega única que simula el rendimiento real en producción.

3. ¿Cómo actúa la parada temprana (Early Stopping) en vuestro código?

Monitorizamos la pérdida de validación (`Val Loss`) al final de cada época. Si durante `paciencia` épocas consecutivas la pérdida no mejora, se interrumpe el entrenamiento y se restaura el modelo al estado exacto guardado en `copy.deepcopy(modelo.state_dict())` de la mejor época histórica.

4. ¿Por qué el Data Augmentation mejora tanto la generalización?

Porque expande artificialmente la diversidad del dataset sin coste de etiquetado. Al someter las fotos a volteos horizontales, pequeñas rotaciones y variaciones cromáticas, impedimos que la red memorice píxeles concretos y la forzamos a aprender rasgos estructurales invariantes del personaje.

5. ¿Por qué funciona Transfer Learning con ResNet18 si se entrenó con fotos de ImageNet y esto son dibujos?

Porque las primeras capas convolucionales de cualquier red profunda aprenden filtros de bajo nivel universales (detección de bordes, líneas, esquinas, texturas y contrastes de color). Esos elementos visuales básicos son idénticos en fotos reales y en dibujos animados. Congelamos esas capas y reentrenamos solo la capa final para adaptarla a los 10 personajes de Los Simpson.